SESSION 2

Kimin İşi Ne?

Service Katmanı, IoC, Dependency Injection & Strategy Pattern

Umay ŞAMLI

umay.samli@infinitumit.com.tr | samliumay965@gmail.com

İçindekiler

1. **Giriş: İş Mantığı Nereye Yazılır?**
2. **Service Katmanı: Mimarideki Rolü**
3. **IoC: Kontrolün Tersine Dönmesi**
4. **Spring IoC Container & Bean Yaşam Döngüsü**
5. **Dependency Injection Derinlemesine**
6. **Lombok: Boilerplate Koddan Kurtulmak**
7. **TaskService: CRUD İş Mantığı**
8. **Problemi Tanıyalım: if-else Zinciri**
9. **Strategy Pattern: Değişen Davranışı Yönetmek**
10. **Open/Closed Principle (SOLID)**
11. **Pratik**
12. **Özet**

1. Giriş: İş Mantığı Nereye Yazılır?

Geçen oturumda Entity ve Repository'yi gördük. Veriyi nasıl temsil edeceğimizi ve veritabanına nasıl konuşacağımızı öğrendik. Bugün şu soruyu yanıtlayacağız: **iş mantığı nereye yazılır?**

"İş mantığı" soyut bir kavram gibi duyuluyor. Somutlaştıralım:

- "Bir görev tamamlandığında, oluşturulma tarihinden 7 günden fazla geçmişse yöneticiye bildirim gönder."
- "Bir kullanıcı aynı başlıkla ikinci bir görev oluşturamaz."
- "Görev durumu sadece PENDING → IN_PROGRESS → DONE sırasıyla ilerleyebilir."

Bunların hepsi **iş kuralları**. Peki bu kurallar kodun hangi katmanına girer?

Controller'a mı? **Hayır** — Controller'ın işi HTTP isteğini karşılamak. Repository'ye mi? **Kesinlikle hayır** — Repository sadece veritabanıyla konuşur. Cevap: **Service**.

Bu Session'da Öğrenecekleriniz

IoC prensibi ve Spring IoC Container • Dependency Injection mekanizması • Bean yaşam döngüsü • Lombok ile temiz kod • Service katmanının sorumluluğu • Strategy Pattern ve Open/Closed Principle • Controller'a giriş

2. Service Katmanı: Mimarideki Rolü

Geçen hafta çizdiğimiz katmanlı mimari şemasını hatırlayın:

`Controller` → `Service` → `Repository` → `Database`

Service katmanı, **Controller ile Repository arasındaki köprüdür**. Ama sadece bir geçit değildir — **karar mekanizmasıdır**.

Katman	Ne Yapar?
Controller	HTTP isteğini alır, Service'e iletir, yanıtı döner. *Trafik polisi.*
Service	**İş kurallarını uygular, kararlar verir, işlemleri koordine eder.**
Repository	Veritabanı CRUD işlemlerini gerçekleştirir. *Veritabanının tercümanı.*

Service Ne Yapar, Ne Yapmaz?

Service'in İşi	Service'in İşi DEĞİL
Yeni görev oluşturma mantığı	HTTP status code belirlemek
Durum geçiş kurallarını uygulamak	JSON parse/serialize etmek
Birden fazla Repository'yi koordine etmek	SQL sorgusu yazmak
Validasyon kurallarını uygulamak	Request/Response formatını bilmek

DİKKAT:

Service katmanında asla `HttpServletRequest`, `@RequestParam` veya `ResponseEntity` gibi HTTP kavramları bulunmamalıdır. Service, HTTP'nin varlığından haberdar olmamalıdır. Yarın aynı iş mantığını bir message queue veya scheduled job'dan çağırmak istediğinizde Service'in değişmemesi gerekir.

3. IoC: Kontrolün Tersine Dönmesi

Service'i yazmaya başlamadan önce temel bir prensibi anlamamız gerekiyor: **Inversion of Control (IoC)** — Kontrolün Tersine Dönmesi.

Normal Akış: Kontrol Sizde

Normalde bir program yazdığınızda **kontrol sizdedir**. Hangi nesneyi ne zaman oluşturacağınıza, ne zaman çağıracağınıza, ne zaman yok edeceğinize siz karar verirsiniz:

```
// Kontrol bizde --- her seyi biz yonetiyoruz
TaskRepository repo = new TaskRepository();
TaskService service = new TaskService(repo);
Task task = service.createTask("Rapor yaz");
```

Bu küçük projelerde işe yarar. Ama proje büyüdükçe düşünün: 50 sınıf var, her biri 3-4 bağımlılığa ihtiyaç duyuyor, bağımlılıkların bağımlılıkları var... Tüm bu nesneleri doğru sırayla oluşturmak ve birbirine bağlamak kabus olur.

IoC Prensibi: Kontrolü Devret

IoC şunu söylüyor: **nesnelerin oluşturulması ve yönetimi senin işin değil.** Bu sorumluluğu bir **framework'e** (bizim durumumuzda Spring'e) devret. Sen sadece "neye ihtiyacım var" söyle, gerisini framework halletsin.

IoC Olmadan	IoC ile
`new TaskRepository()`	Framework oluşturur
`new TaskService(repo)`	Framework bağlar
Yaşam döngüsünü siz yönetirsiniz	Framework yönetir
50 nesneyi doğru sırayla siz oluşturursunuz	Framework bağımlılık ağacını çözer

IoC bir **prensiptir**, belirli bir teknoloji değildir. Spring bu prensibi **IoC Container** adı verilen bir yapıyla hayata geçirir. Nesne oluşturma, yaşam döngüsü yönetimi ve bağımlılık çözümleme — tüm bunlar JVM içinde çalışan IoC Container tarafından yönetilir.

İPUCU:

IoC'yi şöyle düşünün: bir restoranda yemek sipariş veriyorsunuz. Malzeme almak, pişirmek, servis etmek sizin işiniz değil — sadece "ne istediğinizi" söylüyorsunuz. Restoran (framework) gerisini hallediyor. Kontrol sizden restorana geçti — işte IoC budur.

4. Spring IoC Container & Bean Yaşam Döngüsü

IoC bir prensipti. Şimdi Spring'in bu prensibi **nasıl** uyguladığına bakalım.

Bean Nedir?

Spring IoC Container'ın yönettiği her nesneye **Bean** denir. `@Component`, `@Service`, `@Repository`, `@Controller` ile işaretlenen sınıflar birer bean'dir.

Annotation	Anlamı
`@Component`	Genel amaçlı bean. "Bu sınıfı Spring yönetsin."
`@Service`	Service katmanı bean'i. `@Component`'in özelleşmiş hali.
`@Repository`	Repository katmanı bean'i. Veritabanı exception çevirimi yapar.
`@Controller`	Web katmanı bean'i. HTTP isteklerini karşılar.

Teknik olarak `@Service`, `@Repository` ve `@Controller`'ın hepsi aslında birer `@Component`'tır. Spring içinde aynı şekilde bean olarak kaydedilirler. Fark **anlamsal**dır — kodunuzu okuyan kişi "bu bir Service" veya "bu bir Repository" olduğunu hemen anlar.

Bean Yaşam Döngüsü

1. **Uygulama başlar** — Spring, classpath'teki tüm `@Component` türevlerini tarar.
2. **Bean tanımları oluşturulur** — Hangi bean'lerin gerekli olduğu belirlenir.
3. **Bağımlılık çözümleme** — Her bean'in ihtiyacı olan bağımlılıklar belirlenir.
4. **Bean'ler oluşturulur** — Doğru sırayla nesneler yaratılır ve enjekte edilir.
5. **Uygulama çalışır** — Bean'ler hazır, istekler karşılanır.
6. **Uygulama kapanır** — Bean'ler yok edilir.

Bean Scope

Scope	Kaç nesne?	Açıklama
`singleton` (varsayılan)	1	Tüm uygulama boyunca tek bir nesne.
`prototype`	Her istenildiğinde yeni	Her `getBean()` çağrısında yeni nesne.
`request`	Her HTTP istek için 1	Web uygulamalarında.
`session`	Her HTTP session için 1	Web uygulamalarında.

DİKKAT:

Varsayılan scope `singleton`'dır. Bu, `TaskService`'ten uygulama boyunca tek bir nesne olduğu anlamına gelir. Bu yüzden Service sınıflarınızda **durum tutmayın** (instance variable'da kullanıcıya özel veri saklamayın). Farklı kullanıcıların istekleri aynı nesne üzerinden geçer!

5. Dependency Injection Derinlemesine

IoC "kontrolü devret" diyordu. Peki Spring, bir bean'in ihtiyaç duyduğu bağımlılıkları ona **nasıl** veriyor? İşte bu mekanizmaya **Dependency Injection (DI)** denir.

Bir sınıfın çalışması için başka bir sınıfa ihtiyacı varsa, bu ihtiyaç bir **bağımlılıktır** (dependency). Örneğin `TaskService`, görevleri kaydetmek için `TaskRepository`'ye bağımlıdır.

Yanlış Yol: Bağımlılığı Kendin Oluşturmak

```
// BUNU YAPMAYIN
public class TaskService {
    private TaskRepository taskRepository = new TaskRepository();
    // TaskRepository'nin constructor'i degisirse?
    // Test yazarken mock nasil vereceksiniz?
}
```

Bu yaklaşım sınıfları birbirine sıkı sıkıya bağlar (tight coupling). Test edilemez, değiştirilemez, yönetilemez.

Doğru Yol: Bağımlılığı Dışarıdan Almak

```
// DOGRU YOL: Constructor Injection
@Service
public class TaskService {
    private final TaskRepository taskRepository;

    // Bagimlilik disaridan veriliyor
    public TaskService(TaskRepository taskRepository) {
        this.taskRepository = taskRepository;
    }
}
```

`TaskService`, `TaskRepository`'nin nasıl oluşturulduğunu bilmiyor. Sadece "bana bir `TaskRepository` ver" diyor. **Kim veriyor?** Spring IoC Container.

Üç Injection Yöntemi

Yöntem	Nasıl?
Constructor Injection	Constructor parametresi ile
Field Injection	`@Autowired` direkt alana
Setter Injection	Setter metodu ile

1. Constructor Injection (Önerilen)

En temiz yöntemdir. Bağımlılık `final` yapılabilir, test için mock vermek kolaydır. Spring Boot'ta tek constructor varsa `@Autowired` yazmaya bile gerek yoktur.

```
@Service
public class TaskService {
```

```
    private final TaskRepository taskRepository;

    // Spring, tek constructor varsa @Autowired olmadan da enjekte
        eder
    public TaskService(TaskRepository taskRepository) {
        this.taskRepository = taskRepository;
    }
}
```

2. Field Injection (Önerilmez)

`@Autowired` ile doğrudan alana enjekte edilir. Test için mock vermek zordur.

```
@Service
public class TaskService {
    @Autowired  // Spring dogrudan alana enjekte eder
    private TaskRepository taskRepository;
}
```

3. Setter Injection

Opsiyonel bağımlılıklar için kullanılır. Bağımlılık sonradan değiştirilebilir — bu hem avantaj hem dezavantajdır.

```
@Service
public class TaskService {
    private TaskRepository taskRepository;

    @Autowired
    public void setTaskRepository(TaskRepository taskRepository) {
        this.taskRepository = taskRepository;
    }
}
```

6. Lombok: Boilerplate Koddan Kurtulmak

Az önce gördüğümüz constructor injection temiz bir yöntem. Ama her sınıf için getter, setter, constructor tekrar tekrar yazmak yorucu. **Lombok**, derleme zamanında bu kodları otomatik üretir.

Temel Lombok Annotation'ları

Annotation	Ne Üretir?
`@Getter`	Tüm alanlar için getter metodları
`@Setter`	Tüm alanlar için setter metodları
`@NoArgsConstructor`	Parametresiz constructor
`@AllArgsConstructor`	Tüm alanları alan constructor
`@RequiredArgsConstructor`	Sadece `final` alanlar için constructor
`@Data`	Getter + Setter + ToString + EqualsAndHashCode + RequiredArgsConstructor
`@Builder`	Builder pattern ile nesne oluşturma

Entity'de Lombok: @Getter ve @Setter

Projemizdeki `Task` entity'sinde Lombok şöyle kullanılıyor:

```
@Getter
@Setter
@Entity
@Table(name = "tasks")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @Enumerated(EnumType.STRING)
    private TaskStatus status;

    private LocalDateTime createdAt;
}
```

Manuel getter/setter yazmaya gerek kalmadı. Lombok derleme zamanında hepsini üretir.

Entity Annotation'larını Anlayalım

Bu kodda Lombok dışında birkaç JPA annotation'ı var. Her birinin ne yaptığını tek tek inceleyelim.

`@Id` — Bu alanın tablonun **primary key**'i olduğunu belirtir. Her Entity'de mutlaka bir `@Id` olmalıdır. Bu olmadan Hibernate entity'yi tanımaz.

`@GeneratedValue` — ID değerinin **otomatik üretileceğini** söyler. İki parametresi var:

- `strategy`: ID'nin **nasıl** üretileceğini belirler. Seçenekler:

Strateji	Davranış	Kullanım
`IDENTITY`	Veritabanının auto-increment özelliğini kullanır	H2, MySQL
`SEQUENCE`	Veritabanında ayrı bir sequence objesi oluşturur	PostgreSQL, Oracle
`TABLE`	Ayrı bir tablo ile ID üretir	Nadiren kullanılır
`AUTO`	Hibernate veritabanına göre karar verir	Önerilmez

- `generator`: Hangi `@SequenceGenerator` tanımını kullanacağını belirtir. Sadece `SEQUENCE` stratejisinde gereklidir.

Projemizde H2 veritabanı kullanıyoruz ve `GenerationType.IDENTITY` yeterli. Bu, H2'nin auto-increment özelliğini kullanır:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

`@Enumerated(EnumType.STRING)` — Enum değerinin veritabanına **nasıl** kaydedileceğini belirler. İki seçenek var:

Seçenek	DB'de saklanan	Örnek
`EnumType.STRING`	Metin	`"PENDING"`, `"DONE"`
`EnumType.ORDINAL` (varsayılan)	Sıra numarası	`0`, `1`, `2`

DİKKAT:

`@Enumerated` yazmasanız Hibernate varsayılan olarak `ORDINAL` kullanır — yani enum'ları sıra numarası (0, 1, 2) olarak kaydeder. Bu neden tehlikelidir? Diyelim enum şu sırada:

```
public enum TaskStatus {
    PENDING,        // ordinal = 0
    IN_PROGRESS,    // ordinal = 1
    DONE            // ordinal = 2
}
```

Yarın enum'a `CANCELLED` eklediniz ve `DONE`'dan önce koydunuz:

```
public enum TaskStatus {
    PENDING,        // 0
    IN_PROGRESS,    // 1
    CANCELLED,      // 2  --- yeni eklendi
    DONE            // 3  --- eskiden 2'ydi!
}
```

`ORDINAL` kullansaydınız, veritabanında `2` olarak kayıtlı tüm `DONE` satırları artık `CANCELLED` olarak okunurdu. Veri sessizce bozulur, hata bile almadan. `STRING` ile `"DONE"` her zaman `"DONE"` kalır — enum sırası değişse bile. Bu yüzden her zaman `EnumType.STRING` kullanın.

@RequiredArgsConstructor ile DI

`@RequiredArgsConstructor`, `final` alanlar için otomatik constructor üretir. Bu, constructor injection'ı tek satıra indirir:

```
// Lombok OLMADAN
@Service
public class TaskService {
    private final TaskRepository taskRepository;

    public TaskService(TaskRepository taskRepository) {
        this.taskRepository = taskRepository;
    }
}

// Lombok ILE --- ayni sey!
@Service
@RequiredArgsConstructor
public class TaskService {
    private final TaskRepository taskRepository;
    // Constructor otomatik uretilir
}
```

7. TaskService: CRUD İş Mantığı

Şimdi tüm bu bilgileri birleştirip `TaskService`'i yazalım. Projemizdeki gerçek kod:

```java
@Service
@RequiredArgsConstructor
public class TaskService {

    private final TaskRepository taskRepository;
    private final Map<String, TaskSortStrategy> sortStrategies;

    public Task createTask(String title) {
        Task task = new Task();
        task.setTitle(title);
        task.setStatus(TaskStatus.PENDING);
        task.setCreatedAt(LocalDateTime.now());
        return taskRepository.save(task);
    }

    public Task getTaskById(Long id) {
        return taskRepository.findById(id)
                .orElseThrow(() -> new RuntimeException(
                    "Task bulunamadı: " + id));
    }

    public Task completeTask(Long id) {
        Task task = getTaskById(id);
        task.setStatus(TaskStatus.DONE);
        return taskRepository.save(task);
    }

    public List<Task> getAllTasks() {
        return taskRepository.findAll();
    }

    public List<Task> getTasks(String sortType) {
        List<Task> tasks = taskRepository.findAll();

        TaskSortStrategy strategy = sortStrategies
                .getOrDefault(sortType, sortStrategies.get("byDate"
                    ));

        return strategy.sort(tasks);
    }

    public void deleteTask(Long id) {
        Task task = getTaskById(id);
        taskRepository.delete(task);
    }
}
```

Satır Satır İnceleyelim

`@RequiredArgsConstructor`: `final` olan `taskRepository` ve `sortStrategies` için otomatik constructor üretir. Spring bu constructor'ı kullanarak her iki bağımlılığı enjekte eder.

`createTask`: Yeni bir `Task` nesnesi oluşturur, varsayılan değerleri set eder ve kaydeder. Varsayılan durum her zaman `PENDING`'dir — bu bir **iş kuralıdır** ve Service'te uygulanır.

`getTaskById`: Repository'den `Optional<Task>` döner. `Optional`, Java'nın "bu değer var da olmayabilir de" demenin yoludur. Normalde `findById()` bir `Task` dönseydi ve kayıt yoksa `null` gelseydi, siz bunu kontrol etmeyi unuttuğunuzda `NullPointerException` alırdınız. `Optional` sizi bu kontrole **zorlar** — değeri çıkarmadan kullanamazsınız.

```
// YANLIS --- NullPointerException riski
Task task = taskRepository.findById(id).get();

// DOGRU --- Yoksa acikca hata firlat
Task task = taskRepository.findById(id)
    .orElseThrow(() -> new RuntimeException("Task bulunamadi: " +
        id));
```

`orElseThrow` kullanmak, "bu kaydı bulamazsan hata ver" demektir. Böylece `null` kontrolü unutma riski ortadan kalkar.

`completeTask`: Önce görevi bulur (yoksa hata fırlar), sonra durumunu değiştirir ve kaydeder. Dikkat edin: `getTaskById`'yi tekrar kullanıyoruz — **kod tekrarı yok**.

`getTasks`: Sıralama stratejisini Map'ten seçer ve uygular. Bu metodu Strategy Pattern bölümünde detaylı inceleyeceğiz.

`deleteTask`: Önce görevin var olduğunu doğrular (`getTaskById` ile), sonra siler. Olmayan bir görevi silmeye çalışırsanız `getTaskById` hata fırlatır.

completeTask ve Durum Geçişleri

Şu an `completeTask` doğrudan `DONE` yapıyor. Gerçek bir projede durum geçişlerinin kontrol edilmesi gerekir (örneğin PENDING → IN_PROGRESS → DONE sırası). Bu konuyu ilerleyen haftalarda geliştireceğiz.

DİKKAT:

`orElseThrow` içinde `RuntimeException` kullanmak şimdilik yeterli. Ancak gerçek projelerde **özel exception sınıfları** oluşturmalısınız (örneğin `TaskNotFoundException`). Bunu ilerleyen haftalarda geliştireceğiz.

8. Problemi Tanıyalım: if-else Zinciri

Şimdi size gerçek bir problem tanıtmak istiyorum. Vakti geldiğinde hepinizin karşılaşacağı bir durum.

Senaryo: Görevleri listelemek istiyoruz. Ama bazı kullanıcılar tarihe göre, bazıları başlığa göre, bazıları duruma göre sıralanmış liste istiyor.

İlk akla gelen çözüm:

```java
// BUNU YAZMAYIN
public List<Task> getTasks(String sortType) {
    List<Task> tasks = taskRepository.findAll();

    if (sortType.equals("date")) {
        tasks.sort(Comparator.comparing(Task::getCreatedAt));
    } else if (sortType.equals("title")) {
        tasks.sort(Comparator.comparing(Task::getTitle));
    } else if (sortType.equals("status")) {
        // ...
    }
    // Yarin "oncelik" istegi gelirse?
    // Bu metoda tekrar gireceksiniz.
    // Her yeni ozellik mevcut kodu degistirmek demek.
    // Bu tehlikelidir.
    return tasks;
}
```

Bu kod ilk bakışta zararsız görünüyor. Ama sorunları sıralayalım:

1. **Her yeni sıralama = mevcut kodu değiştirmek.** 3 ay sonra 8 farklı sıralama seçeneği eklenince bu metod 50 satıra ulaşır.
2. **Test edilmesi zor.** Her if dalı için ayrı test yazmanız gerekir ve hepsi aynı metod içinde.
3. **Bir dalı değiştirmek diğerlerini kırabilir.** if-else zincirinde bir hata, tüm sıralama mantığını bozar.
4. **Sorumluluk dağınık.** Tüm sıralama stratejileri tek bir metodun içine sıkışmış.

Peki çözüm ne? **Değişen davranışı ayrı sınıflara taşımak.**

9. Strategy Pattern: Değişen Davranışı Yönetmek

Strategy Pattern, **aynı işin farklı yollarla yapılabileceği** durumlarda kullanılır. Fikir basit: değişen davranışı bir arayüz (interface) arkasına sakla. Her strateji kendi sınıfında yaşasın. Yeni strateji eklemek = yeni sınıf yazmak. Mevcut koda dokunmak gerekmesin.

Adım 1 — Ortak Arayüzü Tanımla

```
public interface TaskSortStrategy {
    List<Task> sort(List<Task> tasks);
}
```

Bu interface bir **sözleşme**. "Sıralama yapan her şey bu metodu implemente etmek zorunda" diyor.

Adım 2 — Stratejileri Yaz

```
@Component("byDate")
public class SortByDate implements TaskSortStrategy {
    @Override
    public List<Task> sort(List<Task> tasks) {
        return tasks.stream()
                .sorted(Comparator.comparing(Task::getCreatedAt))
                .toList();
    }
}
```

```
@Component("byTitle")
public class SortByTitle implements TaskSortStrategy {
    @Override
    public List<Task> sort(List<Task> tasks) {
        return tasks.stream()
                .sorted(Comparator.comparing(Task::getTitle))
                .toList();
    }
}
```

Dikkat: `@Component("byDate")` — parantez içindeki isim, bu bean'in **adıdır**. Bu isim birazdan çok önemli olacak.

Adım 3 — Service'e Entegre Et

```
@Service
@RequiredArgsConstructor
public class TaskService {

    private final TaskRepository taskRepository;
    private final Map<String, TaskSortStrategy> sortStrategies;

    public List<Task> getTasks(String sortType) {
```

```
        List<Task> tasks = taskRepository.findAll();

        TaskSortStrategy strategy = sortStrategies
                .getOrDefault(sortType, sortStrategies.get("byDate"
                    ));

        return strategy.sort(tasks);
    }
}
```

Burada Ne Oldu?

Spring'in çok güçlü bir özelliği devreye girdi. `Map<String, TaskSortStrategy>` tipinde bir bağımlılık istediğinizde Spring:

1. `TaskSortStrategy` interface'ini implemente eden tüm bean'leri bulur.
2. Her birini **bean adı** → **bean nesnesi** şeklinde bir `Map`'e koyar.
3. Sonuç: `{"byDate": SortByDate, "byTitle": SortByTitle}`

`getOrDefault` ile geçersiz bir sortType gelirse varsayılan olarak `byDate` stratejisini kullanıyoruz.

Spring'in Map Injection Mekanizması

Spring, bir interface tipinde `Map<String, T>` veya `List<T>` istediğinizde, o interface'i implemente eden **tüm bean'leri** otomatik olarak toplar. Map'te key olarak bean adı, List'te ise tüm bean'ler sırayla gelir. Bu özellik Strategy Pattern'i Spring'de çok doğal hale getirir.

Yeni Strateji Eklemek

Yarın "önceliğe göre sırala" istegi gelse ne yaparsınız?

```
@Component("byPriority")
public class SortByPriority implements TaskSortStrategy {
    @Override
    public List<Task> sort(List<Task> tasks) {
        return tasks.stream()
                .sorted(Comparator.comparing(Task::getPriority))
                .toList();
    }
}
// BITTI. TaskService'e TEK SATIR dokunmadik.
```

Spring, yeni `@Component`'i otomatik olarak `sortStrategies` Map'ine ekler. Service kodu değişmez, test edilmiş kod bozulmaz.

Strategy Pattern'in Spring Boot ile kullanımını daha detaylı incelemek isterseniz [1] kaynağına bakabilirsiniz.

10. Open/Closed Principle (SOLID)

Az önce yaptığımız şeyin yazılım dünyasında bir adı var: **Open/Closed Principle** (Açık/Kapalı İlkesi). SOLID prensiplerinin ikincisi.

Open/Closed Principle

Kod **genişlemeye açık**, **değişime kapalı** olmalıdır.

Yeni özellik eklemek için mevcut kodu değiştirmenize gerek kalmaması gerekir.

if-else Yaklaşımı	Strategy Pattern
Yeni sıralama = mevcut metodu değiştir	Yeni sıralama = yeni sınıf yaz
Test edilen kod bozulabilir	Mevcut kod dokunulmaz
Değişime **açık** (kötü)	Değişime **kapalı** (iyi)
Genişleme zor	Genişlemeye **açık** (iyi)

SOLID'in Diğer Prensipleri

SOLID, beş temel tasarım prensibinin baş harflerinden oluşur. Strategy Pattern'imiz bunların birkaçını aynı anda uyguluyor. Her birini kısaca tanıyalım.

S — Single Responsibility Principle (Tek Sorumluluk)

Her sınıfın **tek bir değişme sebebi** olmalıdır. Bir sınıf birden fazla iş yapıyorsa, bir işi değiştirirken diğerini kırma riskiniz vardır.

```java
// YANLIS --- Hem siralama hem loglama yapiyor
public class SortByDate implements TaskSortStrategy {
    public List<Task> sort(List<Task> tasks) {
        System.out.println("Siralama basladi"); // Log islemi
        return tasks.stream()
                .sorted(Comparator.comparing(Task::getCreatedAt))
                .toList();
    }
}

// DOGRU --- Sadece siralama yapiyor
public class SortByDate implements TaskSortStrategy {
    public List<Task> sort(List<Task> tasks) {
        return tasks.stream()
                .sorted(Comparator.comparing(Task::getCreatedAt))
                .toList();
    }
}
```

O — Open/Closed Principle (Açık/Kapalı)

Yukarıda detaylı işledik: yeni strateji eklemek = yeni sınıf yazmak, `TaskService`'e dokunmamak.

L — Liskov Substitution Principle (Yerine Geçebilirlik)

Bir alt sınıf, üst sınıfın yerine geçtiğinde program bozulmamalıdır. Yani `TaskSortStrategy` bekleyen her yere `SortByDate`, `SortByTitle` veya başka herhangi bir strateji verebilirsiniz — program aynı şekilde çalışır.

```
// Her ikisi de TaskSortStrategy yerine gecebilir
TaskSortStrategy s1 = new SortByDate();
TaskSortStrategy s2 = new SortByTitle();

// Ikisi de ayni sekilde calisir --- program bozulmaz
s1.sort(tasks);
s2.sort(tasks);
```

I — Interface Segregation Principle (Arayüz Ayırma)

Sınıflar, kullanmadıkları metodlara bağımlı olmaya zorlanmamalıdır. Büyük bir interface yerine küçük, odaklı interface'ler tercih edilmelidir.

```
// YANLIS --- Cok fazla sorumluluk tek interface'de
public interface TaskOperations {
    List<Task> sort(List<Task> tasks);
    Task save(Task task);
    void sendEmail(String to);
}

// DOGRU --- Her interface tek bir is yapiyor
public interface TaskSortStrategy {
    List<Task> sort(List<Task> tasks);
}
```

Bizim `TaskSortStrategy` interface'imiz tek bir metod içeriyor — tam da olması gerektiği gibi.

D — Dependency Inversion Principle (Bağımlılık Tersine Çevirme)

Üst seviye modüller, alt seviye modüllere değil **soyutlamalara** (interface) bağımlı olmalıdır.

```
// YANLIS --- Somut sinifa bagimli
public class TaskService {
    private final SortByDate sorter = new SortByDate();
}

// DOGRU --- Interface'e bagimli
public class TaskService {
    private final Map<String, TaskSortStrategy> sortStrategies;
    // Hangi somut sinif gelirse gelsin calisir
}
```

`TaskService`, `SortByDate` veya `SortByTitle`'ı bilmiyor — sadece `TaskSortStrategy` interface'ini biliyor. Yarın yeni bir strateji gelse Service'in haberi bile olmaz.

11. Pratik

Uygulamak isteyene ödev olarak:

Görev 1: TaskService'i Tamamlayın

`TaskService` sınıfını `@RequiredArgsConstructor` ile yazın. Tüm CRUD metodlarını (`createTask`, `getTaskById`, `completeTask`, `getAllTasks`, `deleteTask`) implemente edin.

Görev 2: SortByStatus Stratejisi

`SortByStatus` adında üçüncü bir strateji yazın. Sıralama mantığı: önce `PENDING`, sonra `IN_PROGRESS`, en son `DONE`.

```
@Component("byStatus")
public class SortByStatus implements TaskSortStrategy {
    @Override
    public List<Task> sort(List<Task> tasks) {
        // Buraya siz yazin!
        // Ipucu: TaskStatus enum'inda PENDING=0, IN_PROGRESS=1,
           DONE=2
    }
}
```

Bunu doğru yazdığınızda `TaskService`'e **tek satır eklemeden** yeni strateji çalışıyor olacak.

12. Özet

Konu	Öğrenilen
Service Katmanı	İş mantığının evidir. Controller ile Repository arasındaki koordinatör. HTTP kavramları burada bulunmaz.
IoC Prensibi	Nesne oluşturma ve yönetme kontrolü framework'e devredilir. Spring bunu IoC Container ile uygular.
Dependency Injection	Bağımlılıklar dışarıdan verilir. Constructor Injection en temiz yöntemdir. `final` + `@RequiredArgsConstructor`.
Spring Beans	`@Component` türevleri bean olarak kaydedilir. Varsayılan scope singleton. Durum tutmayın.
Lombok	Boilerplate kodu ortadan kaldırır. Entity'lerde `@Getter`/`@Setter`, Service'lerde `@RequiredArgsConstructor`.
Strategy Pattern	Değişen davranışı ayrı sınıflara taşır. Yeni özellik = yeni sınıf, mevcut kod değişmez. Spring Map injection ile doğal entegrasyon.
SOLID	Beş temel tasarım prensibi. Strategy Pattern bunların birkaçını aynı anda uyguluyor.

Kaynaklar

[1] Implementing the Strategy Design Pattern in Spring Boot, *Medium – Codex*, `https://medium.com/codex/implementing-the-strategy-design-pattern-in-spring-boot-df3adb9ceb4a`